An Agent Based AI Approach to Solving Pacman

Samuel Godwin

K1630575

1. Introduction

The agent built within this project is an intelligent agent, more specifically a simple reflex agent. It interacts with its environment through use of an API. Its percepts are information regarding food and ghosts, and this is acted upon by its movement; eating food as Pacman moves. A simple reflex agent implements actions as a response to what that agent senses in its environment. This environment is a limited, partially observable world sometimes with the presence of ghosts and sometimes without. With ghosts, the world is dynamic due to other processes (the ghosts) having an effect on what happens.

I chose this option as I was particularly inspired by its simplicity, but also because I felt calculating utility values to be unnecessary.

2. Description

2.1. Strategy

To aid in Pacman's navigation of the layout, waypoints are calculated, as described, using information accessed through the API. In his initial situation Pacman will first try to visit all waypoints as this has been designed so it reaches maximum food. It will do this if ghosts are absent or present, and will deviate from this strategy when encountering a ghost that is within the hearing distance of 2 steps away (self-preservation). The waypoints are visited by order of proximity and this will continue after escaping from any ghosts. The path to the waypoint currently closest to Pacman is calculated using an A* search algorithm.

I carefully selected waypoints to be the *turning points* of any layout - that is, a tile that enables a 90-degree turn from any direction (refer to below diagram). This is after a number of observations and calculations in deciding waypoints that would 'see' all food on the map. During their journey to these waypoints, Pacman will build a 'food map' logging the locations of all food observed during the pursuit to visit all waypoints.

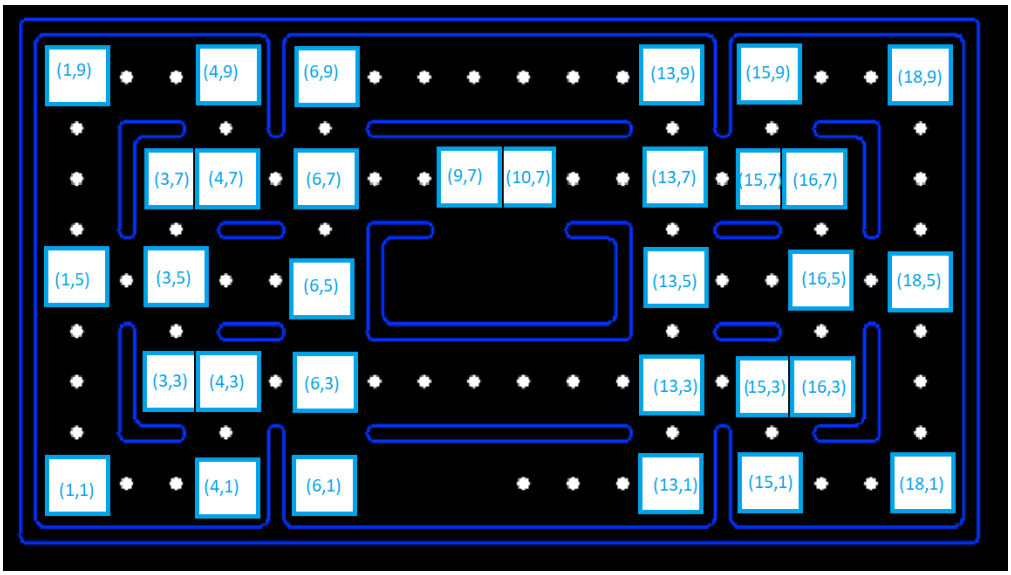

Figure: all points highlighted, with coordinates, on the mediumClassic layout where the ghosts and Pacman can change direction (turning points)

If Pacman finds itself in a situation where a ghost is two steps away, it goes from foraging mode into escape mode, where it purely attempts to run away from ghosts that are in range. This can be directly two steps away or one direct step and one corner step (L-shaped). In order to escape the ghost Pac-Man will take the legal move in the opposite direction to the ghosts

During Pacman's movement around the layout, he will map the observation of food. This utilises his five step view so he can record observed food which he did not reach. Pacman will recognise when food has been consumed and remove it from the map. In the situation where Pac-Man has visited all waypoints he will use this map of remaining food to visit the closet piece of food and continue to ensure he has consumed all food.

Corners- There were certain situations in which Pacman had visited every waypoint but had no spotted the amount of food necessary to finish the game. Generally, this food was out of sight and therefore not added to the food map that is created as Pacman moves. Therefore, a further strategy was needed to ensure Pacman would reach all areas with food in, to notice all food. I noticed several of the layouts had food in the corners and thus, I made use of visiting corners.

As a development of the above corners strategy, if Pacman has reached all corners and still has not found all food, Pacman will take random movements until any remaining food is located or the game ends. Whenever there is food in the food map and Pacman has no

waypoints to pursue, he will always pursue the closest food in the food map (paths calculated with an A* search).

Stop - when Pacman has 2 enemies on either side - Pacman stops for a moment in the hope that the ghosts will either be edible or move in another direction - it is a last defence mechanism that Pacman resorts to when completed trapped.

Remove infinite- I noticed in certain maps that Pacman would fluctuate between two tiles on the map and get stuck in this constantly fluctuating state. I decided that for every step I'd record that in a list of previous steps (I chose seven as a reasonable number of repeats until Pacman would be considered in deadlock) and that list would be checked for fluctuating patterns each time Pacman moves. If this is noticed, the shortest goal Pacman is currently trying to find a path to is removed. This occurs until Pacman is freed from deadlock.

2.2. Methodology

Pac-Man will target the aforementioned waypoints using an A* search. As he travels the layout he will produce a map of remaining food to use once all way points are visited. During this, of course, Pacman will try to escape from ghosts if in danger. If Pacman has visited all waypoints and has visited all food in the food map and the game is still incomplete, Pacman will then visited all corners of the layout.

In my solution I have implemented a "happy forager" state for when Pacman is looking for food and a "worried survivor" state, much like my work in lab practicals. This is inferred through Pacman's behaviour whenever a ghost is near; Pacman flees in whichever opposite direction is available.

I calculated both width and height via the corners accessible through the API. I also created a function to build a map of paths Pacman can walk on for use elsewhere in the program. I built this from information on the layout's walls, which is also accessible through the API.

At the end of the game some state variables are reset to empty so that pacman will fairly win the game each time without relying on information gained in the previous run. When this was not reset, Pac-Man had a very high win rate on the mediumClassic layout (of close to 100%).

The waypoint strategy was favourable since in early iterations of code, my strategy was to simply to eat closest food - this led to index errors, of course because of pacman's limited observability in this version of the API. It resulted in an index limit as it couldn't get the food due to the API's partially observable nature. This did, however, perform well using earlier versions of the API.

Futhermore, my code to pursue the nearest food was, at first, implemented using repeat movements in a random direction (much like RandomishAgent from Lab Practical 1). This also was not useful on the later API. Thus, I introduced an A* search to search for a best path to food. This uses a heuristic of the manhattan distance between the goal of next tile and its neighbours.

Initially, my code to evade the ghosts only functioned when the ghost was on the same X or Y axis, but not when diagonal to Pacman (within two steps but not on the same axis). The strategy was to move in the opposite direction only. Code was added to survey for situations like this and to move in the legal opposite direction (e.g if the ghost was northEast to Pacman, Pacman should move either south or west). From observation it appears that only one option should be available as diagonals usually occur by walls.

It should be noted that Pacman will determine the closest item absolute proximity and not by number of steps on a suitable path. Thus, food just behind a wall (two tiles away) is considered closer than food three direct steps away on the same path Pacman is on. An A* route for the closest item by steps on a suitable path is possible to formulate. This would not greatly enhance performance on the layout mediumClassic, which already has a high success rate, so extra time was not given to developing this feature.

3. Results

3.1. Getting the results

I have used a series of tests to assess my code for functionality under an extensive range of scenarios. To do so, I have started with mediumClassic, although this code was made as general as possible it was designed to fulfil this layout.

I timed mediumClassic (a three segment grid full of food, 2 capsules and an empty small box, and two ghosts) with and then without ghosts to ensure it fulfilled the assignment requirements. This looks for how the code for evading ghosts would affect the overall functionality of the agent.

Following the above, I wished to test my code with the other key layouts which are described below.

In the bigCorners, mediumCorners and tinyCorners layouts, pacman is playing with many turning points and some straight corridors. There are no ghosts or capsules, but food located only in the corners. This layout style is useful for testing the agent's ability to handle lots of turning points (and therefore many waypoint goals for my search function to find). This is also a test of the functionality of the code for finding corners.

The maze layout type features many turning points, no ghosts or capsules, but one piece of food. This ultimately requires pacman to navigate a maze to get to a piece of food that is not visible at most points on the map. It is a good layout for testing pacman's finding of waypoints and getting out of an infinite loop that often arises when trying to navigate to multiple waypoints at a time.

bigSafeSearch is a horizontally long and vertically short grid with lots of food, some capsules that are not close to any food, and one ghost isolated in a closed box. This can be used to see whether pacman will prioritise capsules if there is no food around them, and whether it can see a ghost behind walls. mediumSafeSearch is a smaller version of bigSafeSearch. The ghost is isolated by a larger barrier, which is not expected to affect pacman's behaviour.

bigSearch is a large grid with food in every location, except a centre box. There are no ghosts or capsules. This looks for whether pacman gets stuck in the central box, and ultimately whether visiting all the waypoints will result in all food being collected.

boxSearch is a medium sized box with food spread around in intervals. Ghosts are stuck in a closed corridor to the side of the map. This is another test of food map functionality, due to the absense of waypoints.

capsuleClassic has three ghosts, three capsules, and food items to the sides that are not originally visible to pacman. It demonstrates pacman's ability to escape three ghosts, and continue searching for food when it is initially absent. contestClassic is essentially a larger version of capsuleClassic, it further observes pacman's escaping techniques.

GreedySearch is a small map with no ghosts, no capsules and food around the main corner features. It tests the coding of the legal moves, and pacman's finding of corners.

trappedClassic contains two short corridors, connected by a single turning point. There are two ghosts and four food items in the bottom corridor. This shows pacman's behaviour in close proximity to ghosts with limited space around.

trickySearch is much like mediumSearch and bigSearch, but waypoints are more prominent, and a search for the corners is essential to winning this layout with my agent.

minimaxClassic is another example of ghosts in close proximity to pacman in an enclosed space, while food is located at a turning point for testing of waypoint code.

3.2. Reporting and analysing the results

For all test results, please refer to the below table. All tests were ran 50 times iteratively using **-n 50**.

Name	Average Score	Success Rate (%)
bigCorners:	-578.0 (for all)	50/50 (1.00)
bigMaze	-6.0 (for all)	50/50 (1.00)
bigSafeSearch	1106.0 (all)	50/50 (1.00)
bigSearch	2291.0 (all)	50/50 (1.00)
boxSearch	686.0 (all)	50/50 (1.00)
capsuleClassic	227.96	27/50 (0.54)
contestClassic:	794.26	38/50 (0.76)
contoursMaze:	497.0 (all)	50/50 (1.00)
greedySearch	608.0 (all)	50/50 (1.00)
mediumClassic	1123.7	44/50 (0.88)
mediumClassicNoGhosts	1304.0	50/50 (1.00)
mediumCorners	-1414.4	50/50 (1.00)
mediumDottedMaze	376.0 (all)	50/50 (1.00)
mediumMaze	230.0 (all)	50/50 (1.00)
mediumSafeSearch	770.0 (all)	50/50 (1.00)
mediumScaryMaze	-563.12	5/50 (0.10)
mediumSearch	1330.0 (all)	50/50 (1.00)
minimaxClassic:	-253.76	12/50 (0.24)
oddSearch	892.0 (all)	50/50 (1.00)
openClassic	1203.0	50/50 (1.00)
openMaze	456.0 (all)	50/50 (1.00)
openSearch	1301.0 (all)	50/50 (1.00)
originalClassic	1512.78	26/50 (0.52)
powerClassic	773.28	37/50 (0.74)
smallClassic	748.94	41/50 (0.82)
smallMaze	341.0 (all)	50/50 (1.00)
smallSafeSearch	498.0 (all)	50/50 (1.00)
smallSearch	614.0 (all)	50/50 (1.00)
testClassic	554.36	50/50 (1.00)
testMaze	503.0 (all)	50/50 (1.00)

testSearch	507.0 (all)	50/50 (1.00)
tinyCorners	496.0 (all)	50/50 (1.00)
tinyMaze	492.0	50/50 (1.00)
tinySafeSearch	592.0 (all)	50/50 (1.00)
tinySearch	561.0 (all)	50/50 (1.00)
trappedClassic	-491.96	00/50 (0.00)
trickyClassic	536.06	15/50 (0.30)
trickySearch	540.0 (all)	50/50 (1.00)

On mediumClassic, the default layout, I went a little further and also tested each of 100 runs and 1000 runs in a row. The former I ran multiple times

100 Runs:

Test	Average Score	Success Rate
1	1221.41	93/100 (0.93)
2	1065.33	83/100 (0.83
3	1058.25	82/100 (0.82)
4	1045.4	82/100 (0.82)
5	1051.72	81/100 (0.81)
6	1049.79	80/100 (0.80)

This shows a higher average rate of success with a large number of runs, with none of my 6 tests having a success rate of below 80%.

I also did 1000 runs of mediumClassic, which I ran twice for an average score of 1026.983 and success rate of 796/1000 (0.80) .
This also validates the above observation.

When my code was ran with the mediumClassicNoGhosts it completed the map, won and did so under 30 seconds (when graphics are not used). Similarily, when my code was ran with the mediumClassic it completed the map, won and did so fives times under 2 minutes(when graphics are not used). Thus, when pacman is paying the game on a layout of full food (except small box) and two ghosts he succeeds in the necessary time limits.

I thought I noticed single standalone runs, had a different success rate, to running it in a row. I tested this and recorded that there was no observable difference. This supports that

my code is clearing the global variables properly between runs (and cannot utilise data from previous runs).

From observing pacman in a safe environment such as bigCorners, mediumCorners or tinyCorners, where there are many turning points and very little food, pacman successfully reaches all food 100 percent of the time. Thus, I can conclude that the waypoint strategy was well-written, as evidenced by this high success rate, and is a useful strategy of finding food when it is not always visible to the agent.

From observing pacman in various mazes, I can see that when there are many turning points on the map, pacman can quite easily get 'confused' about which waypoint to move towards. This occurs when the best route to the geographically closest waypoint moves pacman into a position where the next closest waypoint becomes one that is back the way he came from, so he continuously moves between these two routes. This was particularly evident in mediumDottedMaze, in which there was a 100 percent success rate, even with a lower score of 376 each time.

The safe search type layouts were also successful due to the lack of ghosts, and time is also a factor to be considered, as a large score closer to 1000 indicates that it was completed quickly. This suggests that the agent's strategy was sound, eliminating the chance of distraction and wrong moves. The complete lack of effect that the isolated ghost had on pacman shows that the expected result occurred, and pacman could not 'see' the ghost through the closed off walls.

In the bigSearch grid, a consistently very high score of over 2000 demonstrates the importance of finding the right waypoints in a successful solution. However, a longer completion time shows that multiple waypoints in a map significantly increases the time taken for pacman to visit every waypoint.

In the trickySearch layout, my waypoints are all the turning points of the map. All waypoints are searched, but this is not enough to identify the positions of every piece of food, as the lower side of the map is simply a long hallway with food in it at the end. The use of heading towards the corners resolves this issue with searching waypoints in a situation where the food is not visible, and has not been previously stored in the food map.

As indicated in the trappedClassic layout, when surrounded by ghosts with no way out, pacman will stop in his tracks. This is with the purpose of increasing the chance of survival in the main map in hope that a ghost will pass without pursuing pacman.

The minimaxClassic layout shows that chance also plays a significant role in layouts populated with ghosts, as 4 losses and 2 wins were recorded, despite the above-mentioned technique, due to there being too many ghosts and not enough room.

When looking at capsuleClassic and contestClassic, it is clear that evading ghosts is the most important method of a win, as with complete code, the only way a layout can be lost is hitting a ghost.